

Artificial Simulation of Neuronal Evolution

F. Bottega

June 18, 2026

Abstract

This article describes a computational model of digital organisms controlled by evolving artificial brains. Unlike traditional layer-based artificial neural networks, the proposed model represents each brain as a three-dimensional mesh of partially connected neurons, with topology, firing parameters, connectivity regions, and plasticity rules defined by a discrete genome. The organisms inhabit a two-dimensional environment, receive sensory signals, process those signals through a recurrent neuronal dynamics, and execute binary motor actions. Selection is not defined by an explicit loss function; it emerges from a simple ecology based on energy, feeding, survival, and reproduction. We present the formalization of the genome, the rules of phenotypic generation, the signal propagation dynamics, the plasticity of connections, the recombination mechanism through offspring, and the implementation methodology in Godot and Python via WebSocket. In addition to the formal description, the text emphasizes a didactic presentation of the system through flowcharts, diagrams, and tables, making the links between theory, algorithm, and implementation explicit.

Keywords: neuroevolution, genetic algorithms, artificial agents, neuronal plasticity, agent-based simulation, complex systems.

1 Introduction

Traditional artificial neural networks are generally built from pre-specified architectures: layers, blocks, activation functions, matrix operations, and an objective function that guides parameter fitting. This approach is extraordinarily effective in many domains, but it imposes a strongly engineered computational structure. Even when the architecture is large and training is statistical, much of the shape of the cognitive system is defined before experience.

The goal of this work is to explore a complementary direction: to simulate digital organisms whose neuronal structures are not organized in layers, but in a three-dimensional, partially connected mesh that is genetically specified and can be plastically modified throughout life. The methodological hypothesis is not that such a model is immediately competitive with conventional artificial neural networks, but that it offers a suitable experimental space for investigating more organic forms of adaptation, recurrence, memory, self-organization, and environmental selection.

In this model, a digital organism, called here a *blob*, has an artificial brain. The brain is defined by a discrete tensor

$$B = [a] \times [b] \times [c],$$

where $[n] = \{0, 1, \dots, n-1\}$ and $a, b, c \in \mathbb{N}$. Each element of B represents a neuron. Some neurons receive sensory signals, some are associated with motor actions, and the rest function

as internal neurons. Connections between neurons are directional, not necessarily symmetric, and do not follow a layered structure.

The contribution of this text is to formalize the current prototype of the simulation. The article describes:

1. the genetic representation used to generate artificial brains;
2. the genotype–phenotype transformation;
3. neuronal signal dynamics;
4. the rules of structural plasticity and connection weakening;
5. the offspring rules by genetic recombination;
6. the implementation methodology in a Godot frontend and a Python backend.

The current simulation should be understood as an exploratory prototype. In this version, there is no global scalar fitness function, gradient backpropagation, or supervised learning. Selective pressure is environmental: organisms that consume energy without finding food die; organisms that survive long enough and collide with other organisms may generate offspring.

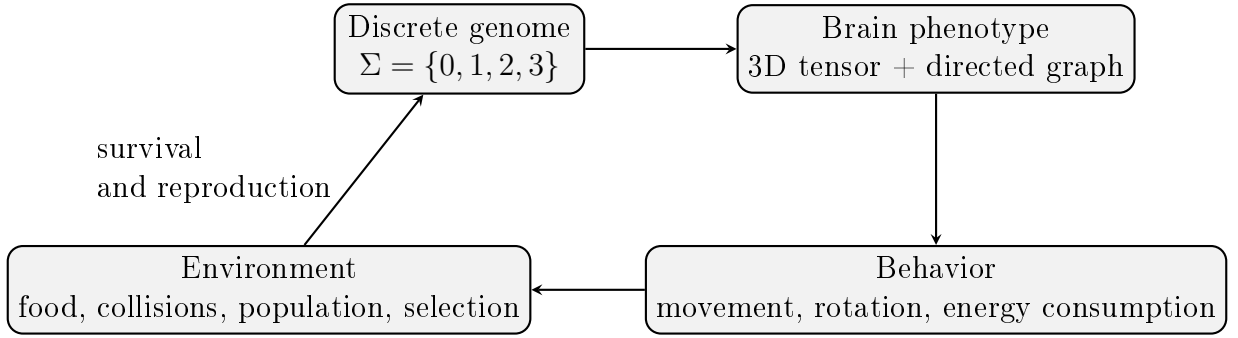


Figure 1: Conceptual cycle of the model: the genome defines the brain, the brain produces behavior, behavior interacts with the environment, and the environment determines which genomes persist.

2 Initial definitions

2.1 Neuronal space

Given dimensions a, b, c , the set of neurons is

$$V_{a,b,c} = [a] \times [b] \times [c].$$

For implementation purposes, each neuron $(i, j, k) \in V_{a,b,c}$ is identified by a linear index

$$\alpha(i, j, k) = i + ja + kab.$$

The set of neuron indices is therefore

$$\mathcal{V}_{a,b,c} = \{0, 1, \dots, abc - 1\}.$$

The inverse transformation is given by

$$x(n) = n \bmod a, \quad y(n) = \left\lfloor \frac{n}{a} \right\rfloor \bmod b, \quad z(n) = \left\lfloor \frac{n}{ab} \right\rfloor.$$

Thus, when convenient, we identify neuron $n \in \mathcal{V}_{a,b,c}$ with the coordinate

$$\alpha^{-1}(n) = (x(n), y(n), z(n)).$$

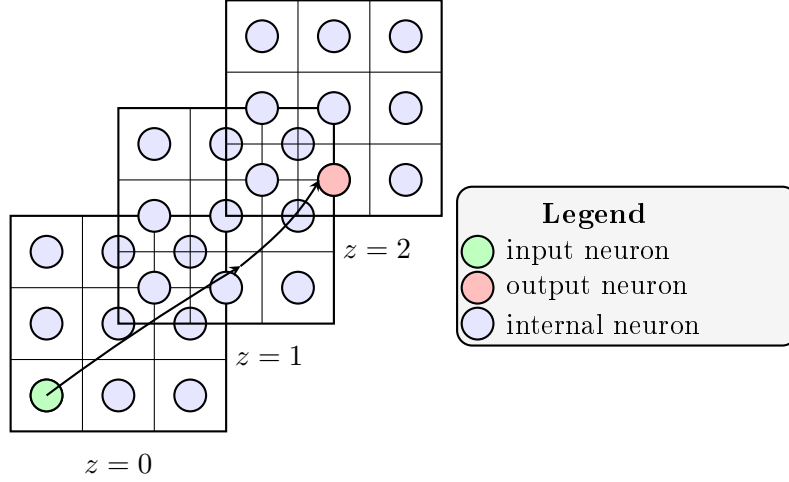


Figure 2: Didactic representation of a brain as a three-dimensional tensor. Each z -plane contains a grid of neurons; some neurons are specialized as inputs and outputs, while the others are internal. The arrows illustrate directional connections.

2.2 Brain graph

The brain of an organism is represented by a directed graph

$$G = (\mathcal{V}_{a,b,c}, E),$$

where $E \subseteq \mathcal{V}_{a,b,c} \times \mathcal{V}_{a,b,c}$ is the set of directed connections. If $(u, v) \in E$, neuron u can send signals to neuron v .

Each connection (u, v) has a synaptic strength

$$\varepsilon_{uv}(t) \in \mathbb{R}_{\geq 0}$$

at discrete time t . Initially, all accepted connections start with strength 1. During the simulation, this strength can increase or decrease according to the usage of the connection.

When a connection weakens below zero, it is removed from the graph. Thus, the brain is not static: its effective graph evolves during the life of the organism.

2.3 Inputs, outputs, and signals

Let N_I be the number of inputs and N_O the number of outputs. The input neurons form a list

$$I = (i_1, \dots, i_{N_I}),$$

and the output neurons form a list

$$O = (o_1, \dots, o_{N_O}).$$

The lists are required to have no repeated indices and, in the current prototype, input and output neurons must be distinct from each other. The internal signal state is stored as a sparse map

$$s_t : \mathcal{V}_{a,b,c} \rightarrow \mathbb{R}_{\geq 0},$$

whose support is finite at each step.

The neuronal state at time t is a sparse function

$$s_t : \mathcal{V}_{a,b,c} \rightarrow \mathbb{R}_{\geq 0}.$$

By convention, if a neuron does not appear in the map, its signal is zero. The external input vector is

$$X_t = (X_t^1, \dots, X_t^{N_I}) \in \mathbb{R}_{\geq 0}^{N_I},$$

and the output vector is

$$Y_t = (Y_t^1, \dots, Y_t^{N_O}) \in \{0, 1\}^{N_O}.$$

The saturation function used to normalize signals is

$$\sigma(x) = 1 - e^{-x/\kappa}.$$

Here κ is a genetically defined parameter that controls how quickly a signal saturates.

Table 1: Basic objects of the model and their interpretation.

Object	Notation	Interpretation
Brain	$B = [a] \times [b] \times [c]$	three-dimensional mesh of neurons
Graph	$G = (\mathcal{V}, E)$	effective directed connectivity structure
Inputs	$I = (i_1, \dots, i_{N_I})$	neurons that receive sensory signals
Outputs	$O = (o_1, \dots, o_{N_O})$	neurons that determine motor actions
Neuronal signal	$s_t(u)$	intensity stored in neuron u at time t
Synaptic strength	$\varepsilon_{uv}(t)$	intensity of the directed connection (u, v)
Internal threshold	θ_u	firing threshold of neuron u
Output threshold	$\Theta_{o_r}^{out}$	threshold for activating the action associated with o_r

3 Genetic representation

3.1 Genetic alphabet

The genome is defined over the discrete alphabet

$$\Sigma = \{0, 1, 2, 3\}.$$

Optionally, these symbols can be interpreted as

$$0 \mapsto A, \quad 1 \mapsto T, \quad 2 \mapsto C, \quad 3 \mapsto G.$$

This interpretation is only symbolic. In the implementation, genes are handled as integers in $\{0, 1, 2, 3\}$.

Given a genetic word

$$w = (w_1, \dots, w_m) \in \Sigma^m,$$

the decimal conversion is defined as

$$D_m(w) = \sum_{\ell=1}^m w_\ell 10^{m-\ell}.$$

Note that D_m does not interpret the word as a number in base 4. It concatenates the symbols as decimal digits restricted to 0, 1, 2, 3. For example,

$$D_5(1, 0, 3, 2, 1) = 10321.$$

The map

$$\mu : \Sigma \rightarrow \{0.2, 0.4, 0.6, 0.8\}, \quad \mu(q) = \frac{q+1}{5}$$

will also be used.

This map converts a genetic symbol into one of the four phenotypic values used as thresholds and probabilities in the model.

3.2 Global parameters

The current implementation uses the constant parameters listed in Table 2.

Table 2: Global parameters of the current model.

Symbol	Value	Meaning
N_I	16	number of sensory inputs
N_O	4	number of motor outputs
D	5	word length for input/output coordinates
D_2	2	word length for regions and local coordinates
D_3	6	word length for the decay gene
D_4	4	word length for the initial number of connections
$A_{\min}, A_{\max}$	4, 7	limits of the first brain dimension
$B_{\min}, B_{\max}$	4, 7	limits of the second brain dimension
$C_{\min}, C_{\max}$	4, 7	limits of the third brain dimension

3.3 Complete genome

The genome of a newly generated organism can be written as

$$\mathcal{G} = (G_{\text{dim}}, G_I, G_O, G_{O,\theta}, G_R, G_Q, G_E, G_P, G_\theta, G_\delta, G_\tau, G_\kappa, G_\omega).$$

Each component encodes one part of the brain phenotype:

Table 3: Genetic components and their associated phenotypes.

Gene	Form	Associated phenotype
G_{dim}	$\Sigma^{3 \times D}$	dimensions (a, b, c)
G_I	$\Sigma^{N_I \times 3D}$	input neurons
G_O	$\Sigma^{N_O \times 3D}$	output neurons
$G_{O,\theta}$	Σ^{N_O}	output thresholds
G_R	$\Sigma^{abc \times 3D_2}$	local connectivity regions
G_Q	Σ^{D_4}	maximum number of initial candidates per neuron
G_E	dictionary of words in Σ^{3D_2}	candidates for initial connections
G_P	Σ^{abc}	probabilities of creating new connections
G_θ	Σ^{abc}	neuronal firing thresholds
G_δ	Σ^{D_3}	synaptic strength decay
G_τ	Σ^{D_2}	post-firing inactivity period
G_κ	Σ^{D_2}	saturation factor of the exponential response
G_ω	Σ^{D_2}	gene reserved for the weakened period

The gene G_ω is kept in the specification because it already exists in the implementation, but its phenotype is not yet used in the current dynamics.

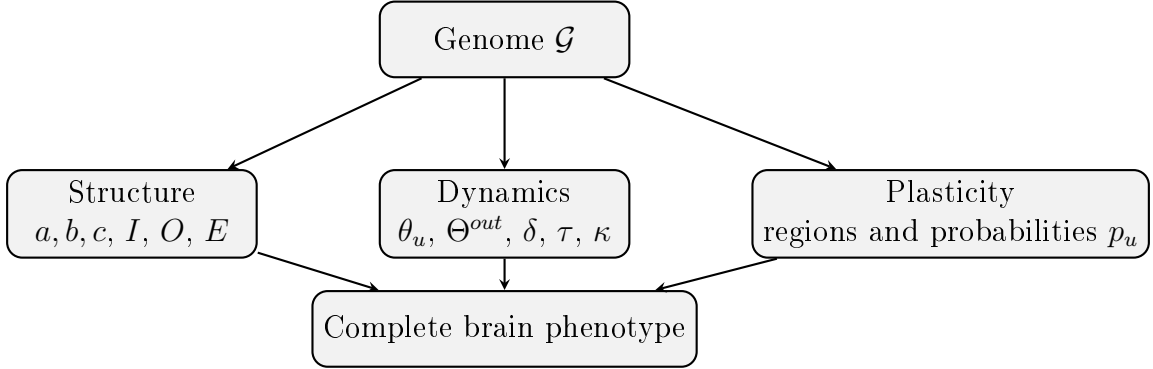


Figure 3: Didactic decomposition of the genome into three functional groups: structural genes, dynamic genes, and plasticity genes.

4 Genetic rules for brain generation

This section describes how a brain is built from a random genome. Generation is rejected and repeated when the structural constraints on inputs and outputs are not satisfied.

4.1 Dimensions

Let

$$G_{\text{dim}} = \begin{pmatrix} g_1^a & \cdots & g_D^a \\ g_1^b & \cdots & g_D^b \\ g_1^c & \cdots & g_D^c \end{pmatrix}.$$

The brain dimensions are

$$a = \max\{A_{\min}, D_D(g_1^a, \dots, g_D^a) \bmod (A_{\max} + 1)\},$$

$$b = \max\{B_{\min}, D_D(g_1^b, \dots, g_D^b) \bmod (B_{\max} + 1)\},$$

$$c = \max\{C_{\min}, D_D(g_1^c, \dots, g_D^c) \bmod (C_{\max} + 1)\}.$$

In the current configuration, $a, b, c \in \{4, 5, 6, 7\}$ and the number of neurons is

$$M = abc.$$

4.2 Genetic mapping to coordinates

Given a word

$$w = (w^x, w^y, w^z) \in \Sigma^{3m},$$

with blocks $w^x, w^y, w^z \in \Sigma^m$, define

$$\Phi_{a,b,c}^{(m)}(w) = \alpha(D_m(w^x) \bmod a, D_m(w^y) \bmod b, D_m(w^z) \bmod c).$$

This function turns a genetic word into a neuron index.

4.3 Input neurons

For each input $r \in \{1, \dots, N_I\}$, the row $G_I[r]$ defines

$$i_r = \Phi_{a,b,c}^{(D)}(G_I[r]).$$

The input list is accepted only if

$$\text{card}\{i_1, \dots, i_{N_I}\} = N_I.$$

If two inputs are mapped to the same neuron, the brain is rejected and a new genome is generated.

4.4 Output neurons

Similarly, for each output $r \in \{1, \dots, N_O\}$,

$$o_r = \Phi_{a,b,c}^{(D)}(G_O[r]).$$

The outputs are accepted only if

$$\text{card}\{o_1, \dots, o_{N_O}\} = N_O$$

and

$$\{o_1, \dots, o_{N_O}\} \cap \{i_1, \dots, i_{N_I}\} = \emptyset.$$

If either condition fails, the brain is rejected.

4.5 Output thresholds

Each output neuron receives a threshold

$$\Theta_{o_r}^{out} = \mu(G_{O,\theta}[r]).$$

Thus,

$$\Theta_{o_r}^{out} \in \{0.2, 0.4, 0.6, 0.8\}.$$

When outputs are read, output r is activated if

$$\sigma_\kappa(s(o_r)) > \Theta_{o_r}^{out}.$$

4.6 Local connectivity regions

Each neuron $u = \alpha(i, j, k)$ has a locally defined rectangular region determined genetically. Let $G_R[u] \in \Sigma^{3D_2}$ be split into three blocks. Define

$$R_x(u) = D_{D_2}(G_R^x[u]) \bmod a,$$

$$R_y(u) = D_{D_2}(G_R^y[u]) \bmod b,$$

$$R_z(u) = D_{D_2}(G_R^z[u]) \bmod c.$$

The nominal connectivity region of u is

$$\mathcal{R}(u) = \{\alpha(p, q, r) : |p - i| \leq R_x(u), |q - j| \leq R_y(u), |r - k| \leq R_z(u)\},$$

with coordinates restricted to the bounds of the brain tensor.

The approximate biological interpretation is that each neuron has its own local reach. There is therefore no single global radius for the whole brain.

4.7 Initial connections

The gene $G_Q \in \Sigma^{D_4}$ determines the number of initial candidates per neuron:

$$Q = D_{D_4}(G_Q) \bmod (M - 1).$$

For each neuron $u = \alpha(i, j, k)$, Q candidate words are generated

$$G_E[u, 1], \dots, G_E[u, Q] \in \Sigma^{3D_2}.$$

Each candidate defines a possible destination

$$v_w = \Phi_{a,b,c}^{(D_2)}(G_E[u, w]).$$

Connection (u, v_w) is added if, and only if, the four conditions below are satisfied:

$$v_w \notin E(u),$$

$$(v_w, u) \notin E,$$

$$v_w \in \mathcal{R}(u),$$

$u \neq v_w$ or, if self-connection is allowed, it does not violate the other constraints.

In the current implementation, the explicit check prevents duplication and reverse connections; self-connections are not treated as a special case. The effective structure is the set of connections accepted after these filters.

4.8 Probability of structural plasticity

Each neuron receives a probability of creating new connections:

$$p_u = \mu(G_P[u]) \in \{0.2, 0.4, 0.6, 0.8\}.$$

This parameter is used during the organism's life, when the neuron fires and tries to create a new connection with another active neuron in its region.

4.9 Neuronal firing threshold

Each neuron receives a firing threshold:

$$\theta_u = \mu(G_\theta[u]) \in \{0.2, 0.4, 0.6, 0.8\}.$$

A neuron u only propagates a signal when

$$s_t(u) > \theta_u$$

and when it is not in its inactivity period.

4.10 Synaptic decay

The decay gene is a word

$$G_\delta = (d_1, \dots, d_{D_3}) \in \Sigma^{D_3}.$$

Define

$$\delta = \sum_{\ell=1}^{D_3} d_\ell 10^{-\ell}.$$

Equivalently, δ is the decimal number $0.d_1d_2\dots d_{D_3}$. Since $d_\ell \in \{0, 1, 2, 3\}$, in the current prototype δ lies in the interval

$$0 \leq \delta \leq 0.333333.$$

At each update, internal connections may have their strength reduced by

$$\delta^2.$$

4.11 Post-firing inactivity

The inactivity period is

$$\tau = D_{D_2}(G_\tau) + 1.$$

This value defines the counter used to prevent a neuron from firing continuously in consecutive iterations. After firing, the neuron enters a refractory state controlled by this counter.

4.12 Exponential factor

The saturation function parameter is

$$\kappa = \left\lfloor \frac{D_{D_2}(G_\kappa)}{10} \right\rfloor + 1.$$

Since $D_{D_2}(G_\kappa)$ ranges from 0 to 33, it follows that

$$\kappa \in \{1, 2, 3, 4\}.$$

4.13 Gene reserved for temporary weakening

The gene

$$G_\omega \in \Sigma^{D_2}$$

defines, in the current implementation, the value

$$\omega = \tau + D_{D_2}(G_\omega) + 1.$$

However, ω does not yet participate in the signal dynamics. It is kept as a reserved parameter for a possible extension in which neurons that have just fired would have a weakened period distinct from the inactive period.

4.14 Genetic generation algorithm

Algorithm 1. New brain generation

1. Sample all genes of $\mathcal{G}$ independently and uniformly in Σ .
2. Compute (a, b, c) from G_{dim} .
3. Build the input neurons I from G_I .
4. If there is a collision between inputs, reject the genome and restart.
5. Build the output neurons O from G_O .
6. If there is a collision between outputs, reject the genome and restart.
7. If $I \cap O \neq \emptyset$, reject the genome and restart.
8. Compute output thresholds, local regions, initial number of candidates, initial connections, plasticity probabilities, internal thresholds, decay, inactivity, and exponential factor.
9. Initialize all neuronal signals as empty.
10. Initialize all existing synaptic strengths as 1.
11. Initialize the inactivity counter of each neuron with value τ .
12. Return the symbolic genome $\mathcal{G}$ and the resulting brain phenotype.

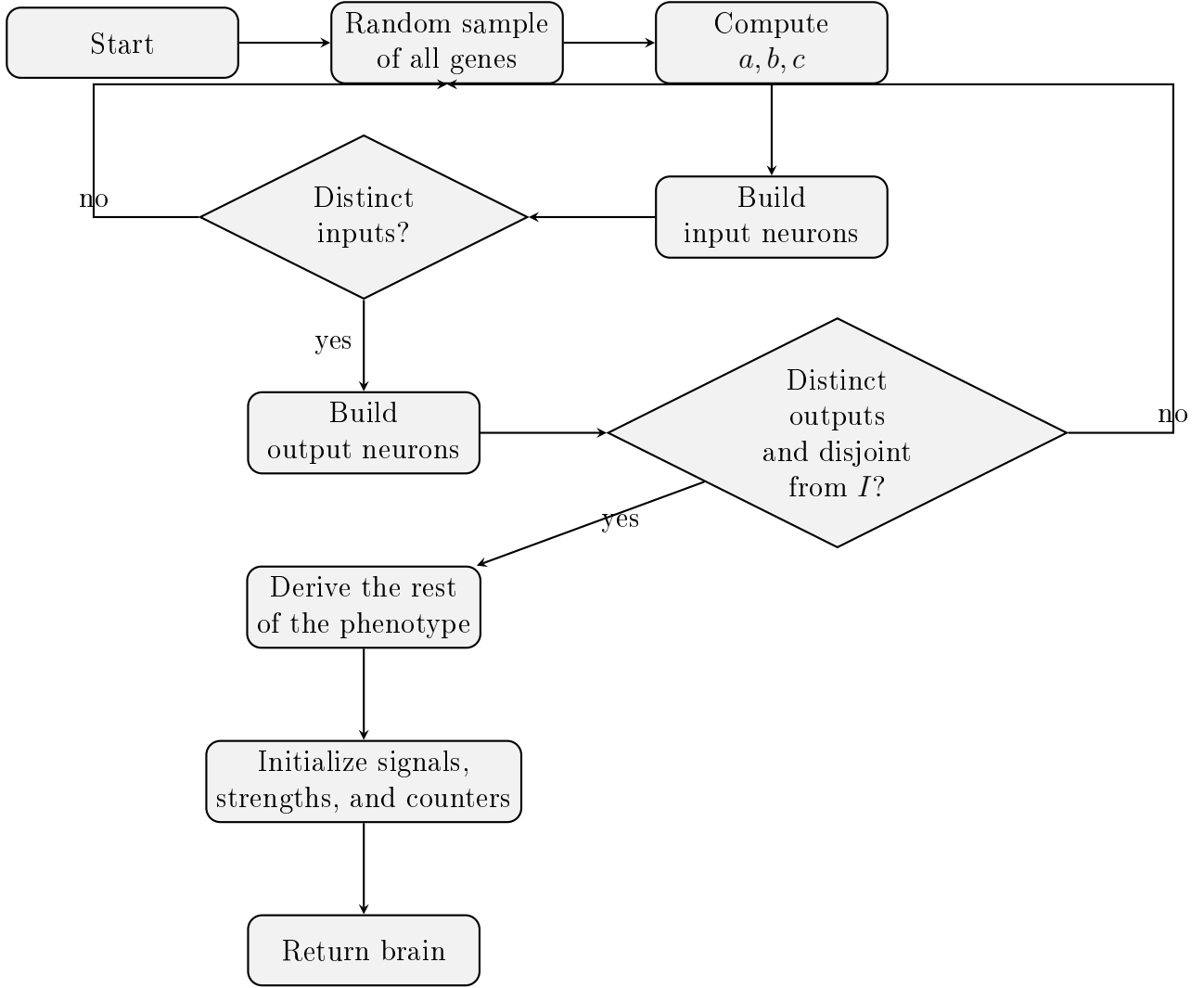


Figure 4: Flowchart of Algorithm 1. Genetic generation is a rejection-sampling procedure: whenever there is a collision between inputs or an invalid separation of inputs/outputs, the genome is discarded and a new one is sampled.

5 Neuronal dynamics

5.1 State structure

For each organism, the simulation keeps:

- the set of connections E_t ;
- the synaptic strengths $\varepsilon_{uv}(t)$;
- the neuronal signals $s_t(u)$;
- the inactivity counters $c_t(u)$;
- the current number of connections.

The processing of a neuronal step receives a vector of external inputs

$$X_t = (X_t^1, \dots, X_t^{N_I}) \in \mathbb{R}_{\geq 0}^{N_I}$$

and returns a binary vector of outputs

$$Y_t = (Y_t^1, \dots, Y_t^{N_O}) \in \{0, 1\}^{N_O}.$$

5.2 Propagation between neurons

Consider a neuron u with signal $s_t(u) > 0$. It fires if

$$s_t(u) > \theta_u$$

and

$$c_t(u) = \tau.$$

When it fires, the effective transmitted signal is

$$\hat{s}_t(u) = \sigma_\kappa(s_t(u)).$$

For each connection $(u, v) \in E_t$, define the usage-updated strength:

$$\tilde{\varepsilon}_{uv}(t) = \frac{1 + \varepsilon_{uv}(t)}{2}.$$

Then

$$\varepsilon_{uv}(t) \leftarrow \tilde{\varepsilon}_{uv}(t)$$

and neuron v receives contribution

$$\Delta s(v) \leftarrow \Delta s(v) + \tilde{\varepsilon}_{uv}(t) \hat{s}_t(u).$$

After transmitting, neuron u is marked for zeroing:

$$s_{t+1}(u) \leftarrow 0,$$

unless it receives another aggregated signal in the same step.

5.3 Structural plasticity

After a firing event, neuron u tries to create a new connection. Let

$$\mathcal{C}_t(u) \subseteq \mathcal{R}(u)$$

be a set of candidates sampled inside the local region of u . In the implementation, for computational economy, the search traverses a sample of approximately half of the points along each axis of the region instead of traversing the full region.

For each candidate $v \in \mathcal{C}_t(u)$, a new connection (u, v) is created if:

$$\begin{aligned} U &\leq p_u, & U &\sim \text{Uniform}(0, 1), \\ v &\in \text{dom}(s_t), \\ s_t(v) &> 0, \end{aligned}$$

$$(u, v) \notin E_t,$$

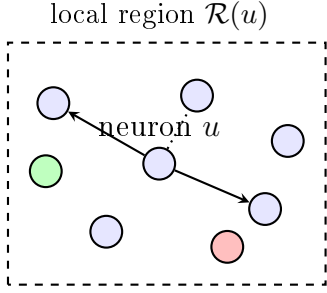
$$(v, u) \notin E_t.$$

When these conditions are satisfied,

$$E_{t+1} \leftarrow E_t \cup \{(u, v)\},$$

$$\varepsilon_{uv}(t+1) = 1.$$

The rule creates at most one new connection per neuron firing.



Didactic reading

Solid lines: existing connections

Dotted line: candidate connection

The candidate is created only if the target neuron is active, lies inside the local region, there is no duplication, and the probabilistic sample succeeds.

Figure 5: Diagram of local structural plasticity. An active neuron tries to create a new connection only inside its genetic connectivity region.

5.4 Inactivity counter update

When a neuron fires, its counter is decreased. While the counter is different from τ , it continues to be decremented at each step. If the counter becomes negative, it is restored to τ . In algorithmic form:

$$c(u) \leftarrow c(u) - 1 \quad \text{on firing,}$$

and, at the end of the neuron evaluation,

$$c(u) \neq \tau \implies c(u) \leftarrow c(u) - 1,$$

$$c(u) < 0 \implies c(u) \leftarrow \tau.$$

This mechanism works as a discrete refractory period. The effective duration depends on τ and on the exact update order.

5.5 Connection weakening

After the firing attempt and structural plasticity, the connections of u are weakened. The current rule preserves all connections involving input or output neurons. Therefore, a connection (u, v) is only decayed if

$$u \notin I \cup O \quad \text{and} \quad v \notin I \cup O.$$

In that case,

$$\varepsilon_{uv}(t+1) = \varepsilon_{uv}(t) - \delta^2.$$

If

$$\varepsilon_{uv}(t+1) < 0,$$

the connection is removed:

$$E_{t+1} \leftarrow E_{t+1} \setminus \{(u, v)\}.$$

This rule implements structural competition: weakly used internal connections tend to disappear, while used connections have their strength driven toward 1 by the rule

$$\varepsilon \mapsto \frac{1 + \varepsilon}{2}.$$

5.6 Injection of external inputs

After internal propagation, external inputs are injected into the input neurons. For each $r \in \{1, \dots, N_I\}$,

$$s(i_r) \leftarrow \sigma_\kappa(X_t^r).$$

Thus, sensory signals are normalized by the same exponential response used in the internal dynamics.

5.7 Output readout

After the input injection, outputs are read. For each $r \in \{1, \dots, N_O\}$,

$$Y_t^r = \begin{cases} 1, & \text{if } o_r \in \text{dom}(s) \text{ and } \sigma_\kappa(s(o_r)) > \Theta_{o_r}^{\text{out}}, \\ 0, & \text{otherwise.} \end{cases}$$

In the current implementation there are four motor outputs:

Table 4: Interpretation of outputs in the two-dimensional environment.

Output	Action
Y^1	forward movement in the direction of the current rotation
Y^2	backward movement relative to the current rotation
Y^3	positive angular rotation
Y^4	negative angular rotation

5.8 Neuronal processing algorithm

Algorithm 2. Processing an organism in one step

1. Receive the external input vector X_t .
2. Create an empty temporary accumulator Δs .
3. Create a list Z of neurons that will be zeroed after firing.
4. For each neuron u currently present in the signal dictionary:
 - (a) If $s(u) > \theta_u$ and $c(u) = \tau$, then:
 - i. compute $\widehat{s}(u) = \sigma_\kappa(s(u))$;
 - ii. add u to list Z ;

- iii. for each connection (u, v) , update $\varepsilon_{uv} \leftarrow (1 + \varepsilon_{uv})/2$ and accumulate $\Delta s(v) \leftarrow \Delta s(v) + \varepsilon_{uv} \widehat{s}(u)$;
 - iv. try to create a new connection starting from u .
 - (b) Update the inactivity counter of u .
 - (c) Apply decay to the internal connections of u .
5. For each $u \in Z$, set $s(u) \leftarrow 0$.
 6. For each v in Δs , set $s(v) \leftarrow \Delta s(v)$.
 7. Inject the external inputs into neurons $i_1, \dots, i_{N_I}$.
 8. Read outputs $o_1, \dots, o_{N_O}$.
 9. Return the binary vector Y_t .

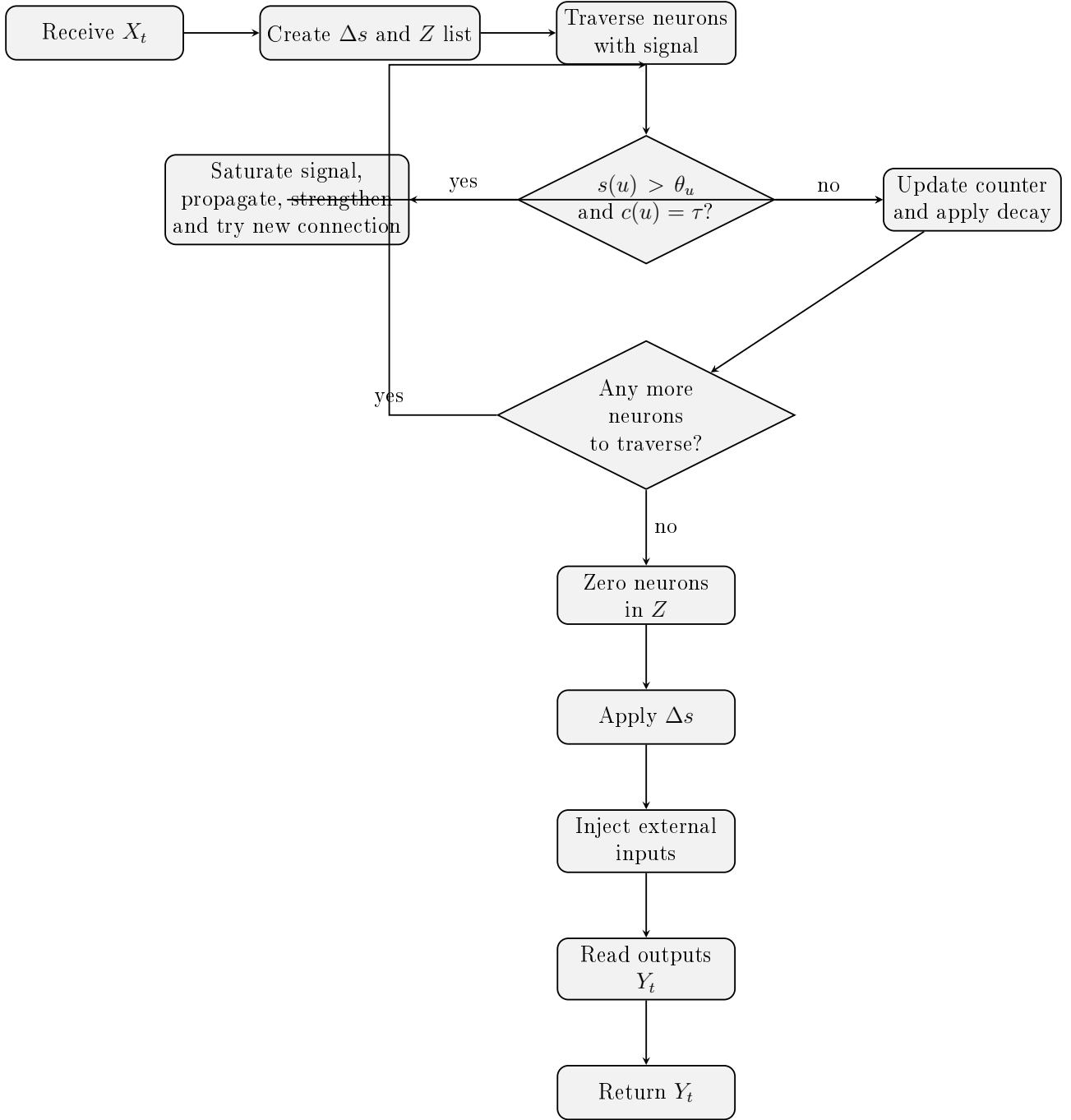


Figure 6: Flowchart of Algorithm 2. Neuronal processing combines internal updating, plasticity, sensory injection, and output readout in a single discrete step.

6 Offspring rules

6.1 Reproductive event

Reproduction is triggered by the environment when two blobs collide and satisfy the behavioral conditions imposed by the simulation: minimum age, child limit, cooldown, and population cap. When reproduction is requested, the backend receives the identifiers of two parent organisms, say P_1 and P_2 .

If either parent is not registered in the set of live organisms maintained by the backend,

the offspring fails. Otherwise, the system tries to generate a child by genetic recombination. In the current prototype, generation is attempted at most twice; if the validity constraints fail repeatedly, no child is returned.

6.2 Child dimensions

Let

$$(a_1, b_1, c_1), \quad (a_2, b_2, c_2)$$

be the parent dimensions. The child dimensions are sampled uniformly from the closed intervals between the parental values:

$$a_h \sim \text{UnifInt}(\min(a_1, a_2), \max(a_1, a_2)),$$

$$b_h \sim \text{UnifInt}(\min(b_1, b_2), \max(b_1, b_2)),$$

$$c_h \sim \text{UnifInt}(\min(c_1, c_2), \max(c_1, c_2)).$$

The number of neurons in the child is

$$M_h = a_h b_h c_h.$$

6.3 Cut operator

To recombine blocks of size L , define a cut point

$$\rho_L \sim \text{UnifInt}(\lfloor 0.4L \rfloor, \lfloor 0.6L \rfloor).$$

The cut-based crossover operation is

$$\text{cross}(A, B; \rho_L)_r = \begin{cases} A_r, & r < \rho_L, \\ B_r, & r \geq \rho_L. \end{cases}$$

The implementation uses this pattern for input genes, output genes, output thresholds, regions, initial connections, probabilities, and internal thresholds, with adaptations when the child has more neurons than the parents.

6.4 Child inputs

Choose

$$\rho_I \sim \text{UnifInt}(\lfloor 0.4N_I \rfloor, \lfloor 0.6N_I \rfloor).$$

The child input gene is

$$G_I^h[r] = \begin{cases} G_I^{P_1}[r], & r < \rho_I, \\ G_I^{P_2}[r], & r \geq \rho_I. \end{cases}$$

Then each row is reinterpreted in the child's dimensions:

$$i_r^h = \Phi_{a_h, b_h, c_h}^{(D)}(G_I^h[r]).$$

The generation is accepted only if

$$\text{card}\{i_1^h, \dots, i_{N_I}^h\} = N_I.$$

Otherwise, a new offspring attempt is started.

6.5 Child outputs

Similarly,

$$\rho_O \sim \text{UnifInt}(\lfloor 0.4N_O \rfloor, \lfloor 0.6N_O \rfloor),$$

and

$$G_O^h[r] = \begin{cases} G_O^{P_1}[r], & r < \rho_O, \\ G_O^{P_2}[r], & r \geq \rho_O. \end{cases}$$

The child output neurons are

$$o_r^h = \Phi_{a_h, b_h, c_h}^{(D)}(G_O^h[r]).$$

The generation is accepted only if

$$\text{card}\{o_1^h, \dots, o_{N_O}^h\} = N_O$$

and

$$\{o_1^h, \dots, o_{N_O}^h\} \cap \{i_1^h, \dots, i_{N_I}^h\} = \emptyset.$$

6.6 Child output thresholds

Choose

$$\rho_{O,\theta} \sim \text{UnifInt}(\lfloor 0.4N_O \rfloor, \lfloor 0.6N_O \rfloor).$$

Then

$$G_{O,\theta}^h[r] = \begin{cases} G_{O,\theta}^{P_1}[r], & r < \rho_{O,\theta}, \\ G_{O,\theta}^{P_2}[r], & r \geq \rho_{O,\theta}. \end{cases}$$

The phenotypic threshold is again

$$\Theta_{o_r^h}^{\text{out},h} = \mu(G_{O,\theta}^h[r]).$$

6.7 Child local regions

The regions are recombined across M_h positions. Choose

$$\rho_R \sim \text{UnifInt}(\lfloor 0.4M_h \rfloor, \lfloor 0.6M_h \rfloor).$$

For each index $u \in \{0, \dots, M_h - 1\}$,

$$G_R^h[u] = \begin{cases} G_R^{P_1}[u], & u < \rho_R \text{ and } u < M_1, \\ G_R^{P_2}[u], & u < \rho_R \text{ and } u \geq M_1 \text{ and } u < M_2, \\ \text{random gene}, & u < \rho_R \text{ and no parental gene is available}, \\ G_R^{P_2}[u], & u \geq \rho_R \text{ and } u < M_2, \\ G_R^{P_1}[u], & u \geq \rho_R \text{ and } u \geq M_2 \text{ and } u < M_1, \\ \text{random gene}, & u \geq \rho_R \text{ and no parental gene is available}. \end{cases}$$

That is: before the cut there is preference for parent P_1 ; after the cut there is preference for parent P_2 ; when the child has neurons beyond the genetic reach of the parents, the gene is filled randomly.

6.8 Child initial number of connections

The gene G_Q^h is chosen entirely from one of the parents:

$$G_Q^h = \begin{cases} G_Q^{P_1}, & U > 0.5, \\ G_Q^{P_2}, & U \leq 0.5, \end{cases} \quad U \sim \text{Uniform}(0, 1).$$

Then

$$Q_h = D_{D_4}(G_Q^h) \bmod (M_h - 1).$$

6.9 Child initial connections

For each neuron u in the child, choose

$$\rho_E \sim \text{UnifInt}(\lfloor 0.4Q_h \rfloor, \lfloor 0.6Q_h \rfloor).$$

For each candidate $w \in \{1, \dots, Q_h\}$, use the preference rule:

$$G_E^h[u, w] = \begin{cases} G_E^{P_1}[u, w], & w < \rho_E \text{ and available,} \\ G_E^{P_2}[u, w], & w < \rho_E \text{ and } P_1 \text{ unavailable but } P_2 \text{ available,} \\ \text{random gene,} & w < \rho_E \text{ and unavailable in both parents,} \\ G_E^{P_2}[u, w], & w \geq \rho_E \text{ and available,} \\ G_E^{P_1}[u, w], & w \geq \rho_E \text{ and } P_2 \text{ unavailable but } P_1 \text{ available,} \\ \text{random gene,} & w \geq \rho_E \text{ and unavailable in both parents.} \end{cases}$$

Each candidate word is reinterpreted in the child's dimensions, and the corresponding connection is accepted if it is not duplicated, if the reverse connection does not exist, and if the destination lies inside the local region of the source neuron.

6.10 Child plasticity probabilities

Choose

$$\rho_P \sim \text{UnifInt}(\lfloor 0.4M_h \rfloor, \lfloor 0.6M_h \rfloor).$$

For each neuron u ,

$$G_P^h[u] = \begin{cases} G_P^{P_1}[u], & u < \rho_P \text{ and available,} \\ G_P^{P_2}[u], & u < \rho_P \text{ and } P_1 \text{ unavailable but } P_2 \text{ available,} \\ \text{random gene,} & u < \rho_P \text{ and unavailable in both parents,} \\ G_P^{P_2}[u], & u \geq \rho_P \text{ and available,} \\ G_P^{P_1}[u], & u \geq \rho_P \text{ and } P_2 \text{ unavailable but } P_1 \text{ available,} \\ \text{random gene,} & u \geq \rho_P \text{ and unavailable in both parents.} \end{cases}$$

The phenotype is

$$p_u^h = \mu(G_P^h[u]).$$

6.11 Child internal thresholds

The recombination of internal thresholds follows the same rule as the probabilities:

$$\rho_\theta \sim \text{UnifInt}(\lfloor 0.4M_h \rfloor, \lfloor 0.6M_h \rfloor),$$

with preference for P_1 before the cut and for P_2 after the cut. The final threshold is

$$\theta_u^h = \mu(G_\theta^h[u]).$$

6.12 Inherited or generated global parameters

The decay is inherited entirely from one of the parents:

$$G_\delta^h = \begin{cases} G_\delta^{P_1}, & U > 0.5, \\ G_\delta^{P_2}, & U \leq 0.5. \end{cases}$$

The post-firing inactivity is generated again at random:

$$G_\tau^h \sim \text{Uniform}(\Sigma^{D_2}), \quad \tau_h = D_{D_2}(G_\tau^h) + 1.$$

The exponential factor is inherited entirely from one of the parents:

$$G_\kappa^h = \begin{cases} G_\kappa^{P_1}, & U > 0.5, \\ G_\kappa^{P_2}, & U \leq 0.5. \end{cases}$$

The reserved gene G_ω is also inherited entirely from one of the parents:

$$G_\omega^h = \begin{cases} G_\omega^{P_1}, & U > 0.5, \\ G_\omega^{P_2}, & U \leq 0.5. \end{cases}$$

6.13 Offspring algorithm

Algorithm 3. Offspring generation

1. Receive parent identifiers P_1 and P_2 .
2. If any parent does not exist in the backend, return failure.
3. Repeat for at most two attempts:
 - (a) sample the child dimensions between the parents' dimensions;
 - (b) recombine input genes with a cut between 40% and 60%;
 - (c) reinterpret inputs in the child dimensions;
 - (d) if there is a collision among inputs, restart the attempt;
 - (e) recombine output genes with a cut between 40% and 60%;
 - (f) reinterpret outputs in the child dimensions;
 - (g) if there is a collision among outputs or an input–output intersection, restart the attempt;

- (h) recombine output thresholds;
 - (i) recombine local regions, using random genes when the child has neurons without parental correspondence;
 - (j) inherit the initial number of connections gene from one of the parents;
 - (k) recombine the initial connection genes for each neuron;
 - (l) recombine plasticity probabilities;
 - (m) recombine internal thresholds;
 - (n) inherit decay, exponential factor, and the reserved weakening gene;
 - (o) generate the inactivity period randomly;
 - (p) initialize signals, synaptic strengths, counters, and number of children.
4. If an attempt is accepted, return the child.
 5. If all attempts fail, return failure.

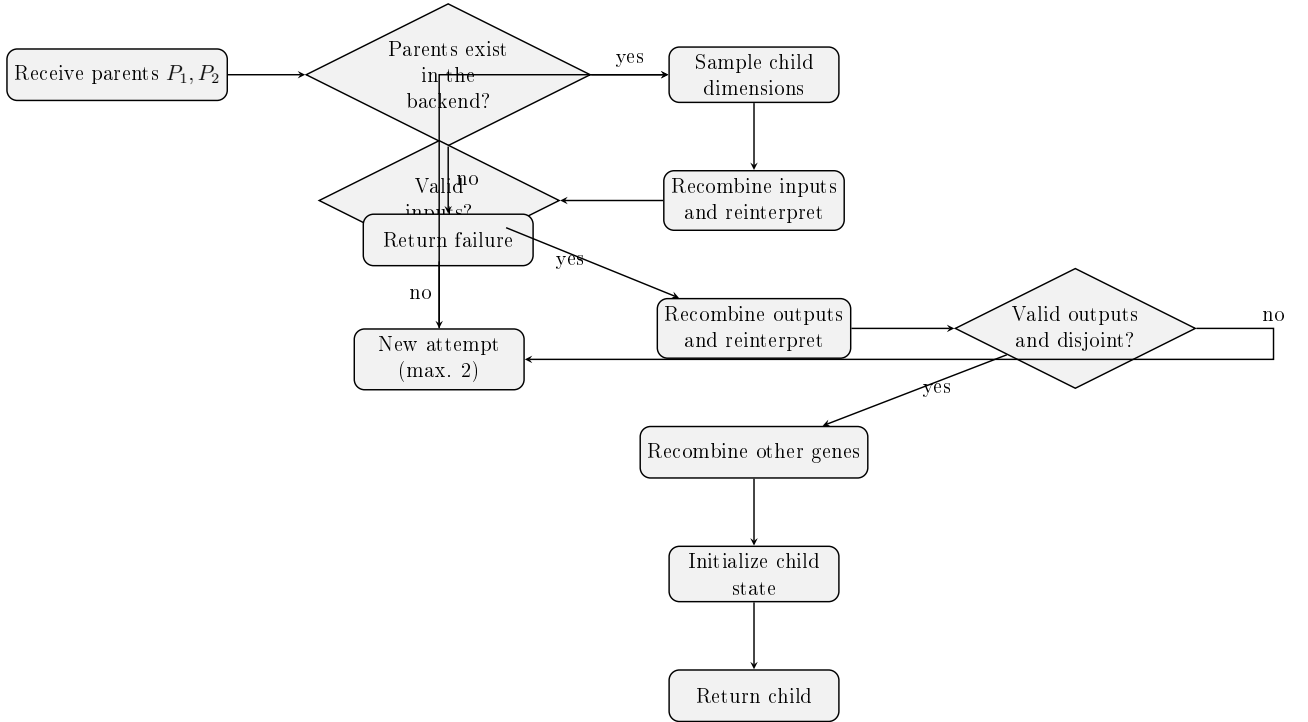


Figure 7: Flowchart of Algorithm 3. The offspring is a recombination with structural validation; whenever the child’s inputs or outputs become invalid, a new attempt is made.

7 Ecology, energy, and selective pressure

The environment acts as a selection mechanism. Each organism has an energy $E_b(t)$, initialized to 1.0. Executing actions consumes energy:

Table 5: Energy costs of the implemented actions.

Action	Condition	Cost
forward movement	$Y^1 = 1$	0.002
backward movement	$Y^2 = 1$ and $Y^1 \neq 1$	0.002
positive rotation	$Y^3 = 1$	0.0015
negative rotation	$Y^4 = 1$ and $Y^3 \neq 1$	0.0015
basal decay	every processed step	0.001
reproduction	offspring event	0.01

When the organism eats, its energy increases:

$$E_b(t+1) = \min\{2.0, E_b(t) + 0.5\}.$$

Reproduction requires a collision between two blobs and the following conditions: minimum age greater than 100 life steps, reproduction cooldown, maximum number of children per blob, and total population below the allowed maximum. The child is born with initial energy 0.5.

There is no explicit fitness. Fitness is implicit and ecological:

$$\text{effective fitness} \approx \text{survive} + \text{find food} + \text{reproduce}.$$

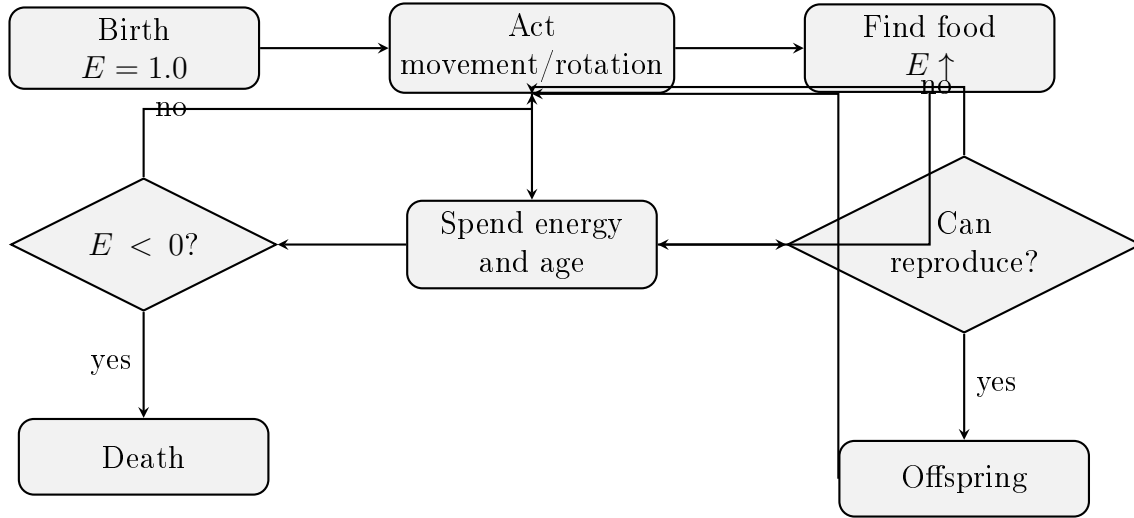


Figure 8: Simplified ecological cycle of the blob: act, spend energy, find food, and eventually reproduce or die.

8 Implementation methodology

8.1 Overall architecture

The implementation is divided into two parts:

1. **Godot frontend:** responsible for the two-dimensional environment, physics, visualization, collisions, food, energy, input collection, and output application.

2. **Python backend:** responsible for brain generation, genome storage, neuronal processing, plasticity, offspring, and communication via WebSocket.

Communication occurs through a local WebSocket at the address

`ws://127.0.0.1:8000/ws`

The backend uses FastAPI and Uvicorn. The frontend uses Godot's `WebSocketPeer`.

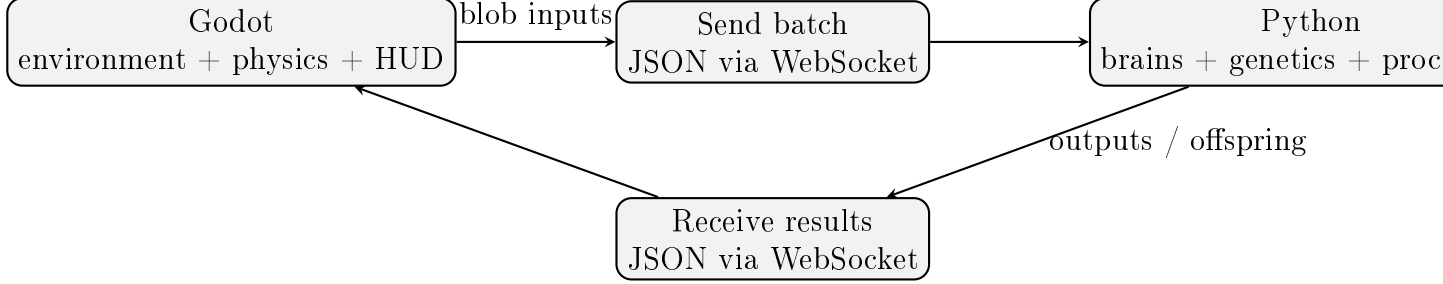


Figure 9: Logical-operational architecture of the implementation. Godot controls the environment and Python controls the brain logic; both communicate in batches via WebSocket.

8.2 Communication protocol

Communication is done in batches. The frontend sends a JSON message of the form

`{"batch": [requests]}`

Each request has a `npc_action` field. The main actions are:

Table 6: Actions supported by the backend.

Action	Function
<code>gen_brain</code>	generate a new brain for a blob
<code>process</code>	process inputs and return outputs
<code>offspring</code>	generate an offspring from two parents

The backend response has the form

`{"results": [results]}`

For processing, each result includes the blob identifier, output vector, current number of connections, and maximum possible brain size.

8.3 Implemented inputs

The blob input vector has 16 components:

$$X = (X_1, \dots, X_{16}).$$

In the current implementation:

$$\begin{aligned}
X_1 &= \max(0, v_x), & X_2 &= \max(0, -v_x), \\
X_3 &= \max(0, v_y), & X_4 &= \max(0, -v_y), \\
X_5 &= \max(0, \omega), & X_6 &= \max(0, -\omega), \\
X_7 &= |\rho/(2\pi)|, & X_8 &= E_b, \\
X_9 &= \frac{1}{1 + E_b}, & X_{10} &= \text{food-proximity counter}, \\
X_{11} &= \text{collision indicator}, & X_{12} &= \min(10, 100/d), \\
X_{13} &= Y_{t-1}^1, & X_{14} &= Y_{t-1}^2, \\
X_{15} &= Y_{t-1}^3, & X_{16} &= Y_{t-1}^4.
\end{aligned}$$

Here v_x, v_y are linear velocity components, ω is the angular velocity, ρ is the current rotation of the blob, E_b is the energy, and d is the distance to a detected food item.

The four previous outputs are fed back as inputs, producing a short-term behavioral memory in the organism–environment coupling.

Table 7: Didactic summary of the 16 inputs.

Input	Description
X_1, X_2	positive and negative horizontal velocity
X_3, X_4	positive and negative vertical velocity
X_5, X_6	positive and negative angular velocity
X_7	normalized rotation
X_8, X_9	energy and regularized inverse energy
X_{10}, X_{12}	proximity and inverse distance to food
X_{11}	collision
$X_{13}, \dots, X_{16}$	previous-step outputs

8.4 Implemented outputs

The binary outputs control body and energy:

1. $Y^1 = 1$: defines forward linear velocity in the direction of the current rotation.
2. $Y^2 = 1$: defines backward linear velocity relative to the rotation.
3. $Y^3 = 1$: defines positive angular rotation.
4. $Y^4 = 1$: defines negative angular rotation.

The physical body of the blob is a gravity-free **RigidBody2D** with linear and angular damping. The scene contains static walls that delimit the environment.

8.5 Food and environmental sensors

Each food item has two detection areas:

1. a larger area, which indicates proximity and provides distance;

2. a smaller area, which represents food consumption.

When the blob enters the larger area, it receives a proximity signal and inverse distance. When it enters the smaller area, the food is removed from the scene and the blob's energy increases. New food items are periodically added to the environment.

8.6 Computational economy

Processing every brain in every frame can become costly. For this reason, the backend uses a cost-saving strategy: the blob with identifier `blob_0` is always processed, while the others are processed in subsets defined by the numeric identifier modulo 5. Thus, each request processes only part of the population, reducing the cost per frame.

The `blob_0` is also used as a reference organism for logging neuronal activity. In the frontend, its energy is kept at 1.0 to prevent it from dying before the simulation ends.

8.7 Histories and visualization

The backend keeps two histories in memory:

1. neuronal signal history;
2. history of the number of connections per blob.

The post-processing visualization script uses Plotly to generate a three-dimensional animation of neuronal activity and a time series of the number of connections. These tools are important for evaluating the structural and dynamic evolution of the brains, even though the current prototype version must ensure consistency between the saved history format and the format expected by the visualization.

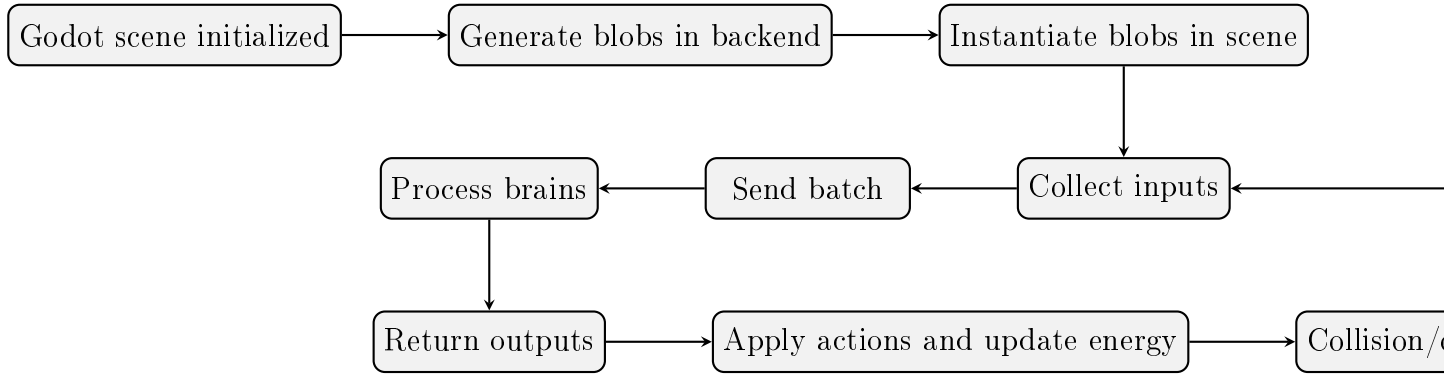


Figure 10: Summary of the operational cycle of the implementation. The system alternates between perception, brain processing, action, and environmental update.

9 Discussion

The proposed model clearly separates three levels of adaptation:

1. **Genetic adaptation:** brain structures are generated and recombined by discrete genes.

2. **Individual adaptation:** internal connections can be created and weakened during the organism’s life.
3. **Ecological adaptation:** the environment selects organisms by survival, feeding, and reproduction.

The absence of layers and of an explicit loss function makes the model less controllable, but also more open. Useful behaviors are not imposed by direct engineering; they must emerge from combinations of topology, signals, plasticity, and selective pressure. This openness is precisely the point of interest of the prototype.

There are, however, important limitations. Some genes already exist in the structure, such as the weakened period, but are not yet used in the dynamics. Such elements should be treated as natural extensions of the model.

Table 8: Summary of the prototype’s hypotheses and implications.

Modeling hypothesis	Experimental implication
Layer-free topology	enables emergent recurrence and local geometry
Discrete genome with indirect mapping	favors structural diversity and rejection of invalid brains
Plasticity during life	separates ontogenetic adaptation from evolutionary adaptation
Selection without explicit fitness	shifts the problem to the environment’s ecology
Distributed backend processing	makes possible brains that are more expensive than full execution inside the graphical engine

10 Conclusion

This article formalized a prototype of artificial neuronal evolution simulation. Each organism has a brain defined by a three-dimensional mesh of neurons, directional connections, thresholds, local connectivity regions, exponential response, structural plasticity, and selective weakening of internal connections. The initial generation of brains is genetic, reproduction occurs through recombination of parental genes, and selection is determined by the environment.

The model does not aim to replace traditional artificial neural networks, but rather to explore an alternative regime of artificial intelligence based on organisms, ecology, plasticity, and evolution. The natural next step is to instrument the histories better, introduce lineage metrics, test different mutation rules, and evaluate whether more adapted neuronal structures emerge robustly under different environmental configurations.